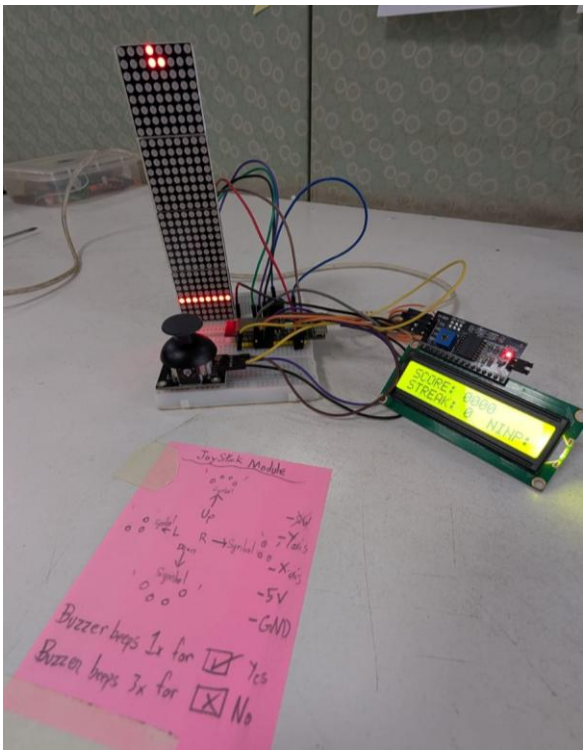
StickShift Hero Playbook



I. Challenge

Build a bare-metal, latency-optimized 1D rhythm arcade system that decouples high-speed human analog input polling from a fixed-interval rendering engine. The system must continuously track a 2-axis analog joystick across calibrated deadzones, buffer user directional flicks without missing inputs during visual rendering steps, render scrolling note patterns on a chained LED dot matrix array, update telemetry on an I^2C LCD dashboard, and execute non-blocking audio feedback without RTOS overhead.

II. Guide

A. Concept

i. Underlying Engineering Principles & Reference Links

1. **Asynchronous Input Buffering (Input Latching):** Human reaction times do not align perfectly with rigid machine frame cycles. Polling joystick ADC values continuously in the main execution loop enables the system to "latch" rapid directional flicks into memory as soon as thresholds are crossed. When the fixed 150 ms game tick fires, it consumes the latched variable, registers the evaluation, and flushes the buffer.
    - *Learn More:* Game Programming Patterns – Event Queue & Input Buffering
2. **Deterministic Time-Sliced Game Loops (millis() State Engine):** Halting CPU execution using rigid delay() functions freezes analog input polling and audio timeout routines. Utilizing a non-blocking millis() state loop ensures game physics, evaluation, and rendering process only when current_time - last_tick_time >= 150ms, preserving deterministic execution.
    - *Learn More:* Adafruit – Multi-tasking the Arduino: Non-blocking Timing Loops
3. **Non-Blocking Asynchronous Audio Control:** Active buzzers require constant DC voltage to oscillate. Rather than stalling the processor to wait for a beep to finish, logging a target buzzer_off_time timestamp allows the main loop to check conditions thousands of times per second and toggle the output pin LOW instantly upon expiration.
    - *Learn More:* Arduino Documentation – Non-Blocking Tone and Tone Timers

4. **Analog Threshold Windowing & Deadzone Calibration:** Potentiometer-based joysticks suffer from mechanical centering drift and noise. Setting discrete software activation thresholds (e.g., $V_Y > 2600$ or $V_Y < 1400$ on a 12-bit ADC) creates a stable deadzone that prevents phantom directional triggers.
    - *Learn More:* Electronic Wings – Analog Joystick Interfacing
5. **Chained Shift Register Display Driving (SPI Matrix):** Displaying graphics on multi-module LED arrays like the MAX7219 involves bit-shifting frame data down a serial daisy chain. Utilizing high-speed hardware SPI communication offloads frame updates with minimal CPU overhead.
    - *Learn More:* Maxim Integrated / Analog Devices – MAX7219 Cascaded Display Architecture

ii. Real-World Example

Arcade rhythm platforms (such as *Dance Dance Revolution*, *Guitar Hero*, or *Beat Saber*), fly-by-wire flight control stick systems, and industrial crane joysticks. These applications rely on high-frequency asynchronous input sampling paired with deterministic processing loops to guarantee zero missed inputs.

B. Components

- **Microcontroller:** WeAct Studio ESP32-C6-N4 Dev Board
- **Primary Display:** 4×1 Chained MAX7219 Dot Matrix Array (32×8 pixels)
- **Telemetry Dashboard:** 1602 LCD with PCF8574 I^2C Backpack
- **Input Device:** 2-Axis Analog Joystick Module
- **Audio Output:** 1× Active Buzzer Module ($3.3V/5V$)
- **Power Supply:** USB-C ($5V@500$ mA minimum)

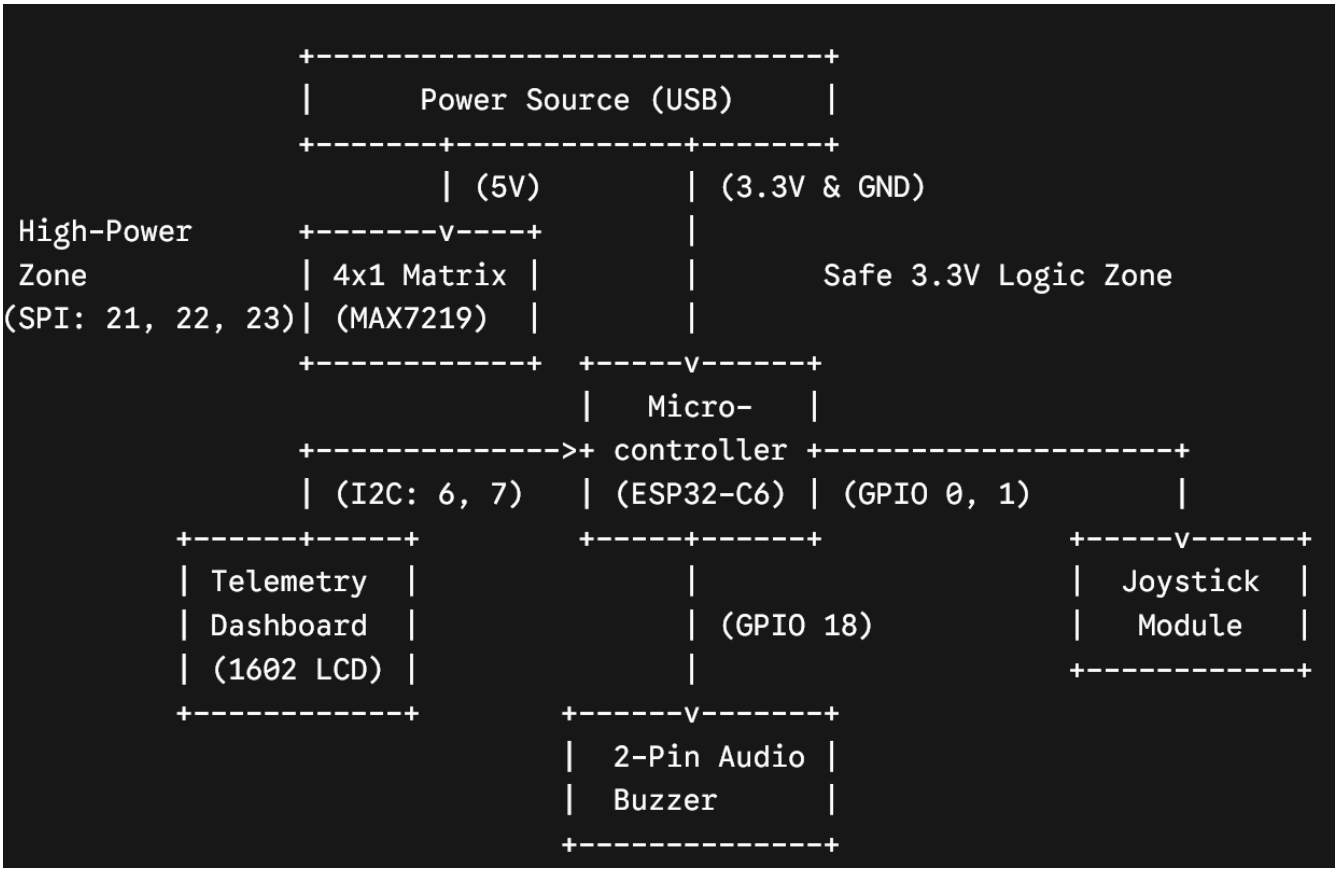
C. Connections

Power Rail Warning: The MAX7219 matrix array and 1602 LCD backlight draw high peak current. They must be powered directly from the $5V$ USB bus rail to avoid thermal overload on the ESP32-C6's internal $3.3V$ LDO regulator.

Subsystem	Component Pin	ESP32-C6 Pin	Signal Type	Functional Description
Power	VCC (Matrix & LCD)	5V	Power Out	Unregulated 5V USB bus power
Power	VCC (Joystick)	3V3	Power Out	Limits analog sweep to safe 3.3V ADC levels
Power	GND (All)	GND	Ground	Common system ground plane

Subsystem	Component Pin	ESP32-C6 Pin	Signal Type	Functional Description
Input	VRX (X-Axis)	GPIO 0	Analog In	12-bit ADC threshold tracking (Left/Right)
Input	VRY (Y-Axis)	GPIO 1	Analog In	12-bit ADC threshold tracking (Up/Down)
Telemetry	SDA	GPIO 6	I^2C Data	Hardware I^2C bus
Telemetry	SCL	GPIO 7	I^2C Clock	Hardware I^2C bus
Audio	Positive (+)	GPIO 18	Digital Out	Non-blocking digital chirp actuation
Matrix	CS	GPIO 21	Digital Out	SPI Slave Select
Matrix	CLK	GPIO 22	SPI Clock	High-speed data clock
Matrix	DIN	GPIO 23	SPI MOSI	Bit-shifted pixel array data

Block Diagram:



Note: If you are working with a 3-pin buzzer, the code does not change but you will need to feed the buzzer with 3.3V power from the development board's 3V3 power pin output.

D. Code

```
C++
/*
 * Project: StickShift Hero (ESP32-C6 Rhythm Arcade System)
 * Hardware: ESP32-C6, MD_MAX72xx Matrix, I2C 1602 LCD, Joystick, Active Buzzer
 */

#include <Arduino.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <MD_MAX72xx.h>

// --- Matrix SPI Configuration ---
#define HARDWARE_TYPE MD_MAX72XX::FC16_HW
#define MAX_DEVICES 4
const int DATA_PIN = 23;
const int CLK_PIN  = 22;
const int CS_PIN   = 21;
MD_MAX72XX mx = MD_MAX72XX(HARDWARE_TYPE, DATA_PIN, CLK_PIN, CS_PIN, MAX_DEVICES);

// --- Telemetry Dashboard Configuration ---
LiquidCrystal_I2C lcd(0x27, 16, 2);

// --- Input & Output Pin Mapping ---
const int JOY_X_PIN  = 0;
const int JOY_Y_PIN  = 1;
const int BUZZER_PIN = 18;

// --- Game Logic Enums & Data Structures ---
enum NoteDirection { NONE = 0, UP = 1, DOWN = 2, LEFT = 3, RIGHT = 4 };
const char* dirStrings[] = { "NONE ", "UP   ", "DOWN ", "LEFT ", "RIGHT" };

// --- Engine Metrics & Deterministic Timing ---
int score = 0;
int streak = 0;
unsigned long last_tick_time = 0;
const unsigned long GAME_TICK_MS = 150;

// --- Volatile Game States ---
int current_note_x = -1;
NoteDirection current_note_dir = NONE;
const int TARGET_LINE_X = 3;

// --- Asynchronous Buffers ---
NoteDirection buffered_input = NONE;
unsigned long buzzer_off_time = 0;
bool buzzer_playing = false;

// --- System Function Declarations ---
NoteDirection readJoystick();
void spawnNote();
void renderGameMatrix();
void triggerAudio(unsigned long duration);
void checkAudioTimeout();
void updateLcdMetrics();
void updateLcdInput(NoteDirection dir);

void setup() {
    Serial.begin(115200);
    analogReadResolution(12);

    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);

    Wire.begin(6, 7);
```

```
    lcd.init();
    lcd.backlight();
    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("SCORE: 0000");
    lcd.setCursor(0, 1);
    lcd.print("STREAK: 0   INP:");

    mx.begin();
    mx.control(MD_MAX72XX::INTENSITY, 2);
    mx.clear();

    spawnNote();
    last_tick_time = millis();
}

void loop() {
    unsigned long current_time = millis();

    // 1. High-Speed Hardware Polling & Audio Management
    checkAudioTimeout();
    NoteDirection current_reading = readJoystick();

    if (current_reading != NONE && current_reading != buffered_input) {
        buffered_input = current_reading;
        updateLcdInput(buffered_input);
    }

    // 2. Strict Deterministic Engine Tick
    if (current_time - last_tick_time >= GAME_TICK_MS) {
        last_tick_time = current_time;
        NoteDirection player_input = buffered_input;

        // Evaluation Phase
        if (current_note_x == TARGET_LINE_X) {
            if (player_input == current_note_dir && player_input != NONE) {
                score += 10;
                streak++;
                current_note_x = -1;
                triggerAudio(30);
            }
        }

        // Physics & Translation Phase
        if (current_note_x >= 0) {
            current_note_x--;
        } else {
            spawnNote();
        }

        // Penalty Phase
        if (current_note_x < TARGET_LINE_X && current_note_x >= 0 && current_note_dir != NONE) {
            streak = 0;
            triggerAudio(120);
        }

        // Render Phase
        renderGameMatrix();
        updateLcdMetrics();

        // Diagnostic Serial Telemetry Stream
        Serial.print("Score:"); Serial.print(score); Serial.print(",");
        Serial.print("Streak:"); Serial.print(streak); Serial.print(",");
        Serial.print("Note_X:"); Serial.print(current_note_x); Serial.print(",");
        Serial.print("Input_Dir:"); Serial.println(player_input);

        // Flush Buffers
        buffered_input = NONE;
        updateLcdInput(NONE);
    }
}
```

```
137.
138. NoteDirection readJoystick() {
139.     int x_val = analogRead(JOY_X_PIN);
140.     int y_val = analogRead(JOY_Y_PIN);
141.
142.     // Actuation bounds for minimized deadzone. Change directional inputs here
143.     if (y_val > 2600) return UP;
144.     if (y_val < 1400) return DOWN;
145.     if (x_val < 1400) return LEFT;
146.     if (x_val > 2600) return RIGHT;
147.
148.     return NONE;
149. }
150.
151. void spawnNote() {
152.     current_note_x = (MAX_DEVICES * 8) - 1;
153.     current_note_dir = (NoteDirection)random(1, 5);
154. }
155.
156. void triggerAudio(unsigned long duration) {
157.     digitalWrite(BUZZER_PIN, HIGH);
158.     buzzer_off_time = millis() + duration;
159.     buzzer_playing = true;
160. }
161.
162. void checkAudioTimeout() {
163.     if (buzzer_playing && millis() >= buzzer_off_time) {
164.         digitalWrite(BUZZER_PIN, LOW);
165.         buzzer_playing = false;
166.     }
167. }
168.
169. void updateLcdMetrics() {
170.     lcd.setCursor(7, 0);
171.     if (score < 1000) lcd.print("0");
172.     if (score < 100)  lcd.print("0");
173.     if (score < 10)   lcd.print("0");
174.     lcd.print(score);
175.
176.     lcd.setCursor(8, 1);
177.     lcd.print(streak);
178.     if (streak < 10) lcd.print(" ");
179. }
180.
181. void updateLcdInput(NoteDirection dir) {
182.     lcd.setCursor(11, 1);
183.     lcd.print(dirStrings[dir]);
184. }
185.
186. void renderGameMatrix() {
187.     mx.clear();
188.     for (int y = 0; y < 8; y++) {
189.         mx.setPoint(y, TARGET_LINE_X, true);
190.     }
191.
192.     if (current_note_x >= 0) {
193.         switch (current_note_dir) {
194.             case UP:
195.                 mx.setPoint(3, current_note_x, true); mx.setPoint(4, current_note_x, true);
196.                 mx.setPoint(2, current_note_x + 1, true); mx.setPoint(5, current_note_x + 1, true);
197.                 break;
198.             case DOWN:
199.                 mx.setPoint(3, current_note_x + 1, true); mx.setPoint(4, current_note_x + 1, true);
200.                 mx.setPoint(2, current_note_x, true); mx.setPoint(5, current_note_x, true);
201.                 break;
202.             case LEFT:
203.                 mx.setPoint(3, current_note_x, true); mx.setPoint(4, current_note_x, true);
204.                 mx.setPoint(3, current_note_x + 1, true);
205.                 break;
206.             case RIGHT:
207.                 mx.setPoint(3, current_note_x, true); mx.setPoint(4, current_note_x, true);
208.                 mx.setPoint(4, current_note_x + 1, true);
```

```
                break;
            default: break;
        }
    }
    mx.update();
}

```

Student Tip: Open the **Serial Monitor** (Ctrl+Shift+M) at **115200 baud** to view score streams and input latching metrics.

To visually inspect engine performance, open the **Serial Plotter** (Ctrl+Shift+L). Graphing Note_X alongside Input_Dir and Streak provides visual verification of latched input timing as note targets cross the trigger line at index 3.

III. Play & Share

- **Adaptive Speed Progression Engine:** Modify GAME_TICK_MS so that as a player builds higher streaks (e.g., every 5 consecutive hits), the game tick interval decreases from 150 ms down to a fast-paced 80 ms.
- **Multi-Note Polyphonic Audio Engine:** Upgrade the active buzzer hardware to a passive piezo speaker driven by ledcWrite() (PWM frequency synthesis). Assign unique musical pitches to each directional arrow (e.g., UP = 440 Hz, DOWN = 330 Hz) so successfully hitting notes generates a melody!
- **Dynamic Combo Multiplier:** Introduce a score multiplier algorithm where maintaining a $10+$ note streak doubles score output ($2\times$), and a $20+$ note streak triples it ($3\times$), complete with special flashing animations on the 1602 LCD screen.
- **Alternative Control Scheme:** Can you switch the joystick module in favor of push buttons instead? Or mess with your friends by secretly shuffling the control scheme!
- **Alternative Games:** What other games can you think of when you use the MAX 7219 Dot Matrix Display? Some suggestions could be Pong, Tetris or even Snake!